



“We don’t just sit around and wait for other people. We just make, and we do.”

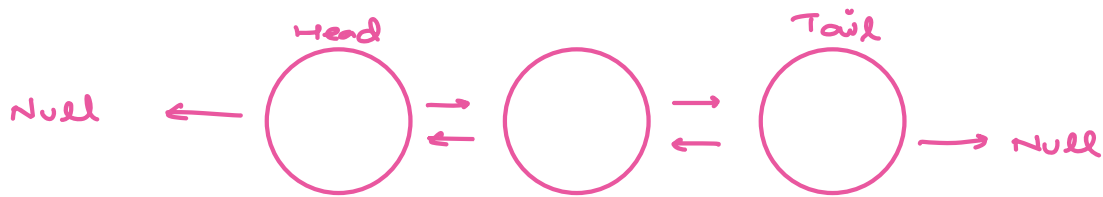
Arlan Hamilton

Today's content

01. Doubly linkedlist
02. Insert a new node just before tail
03. Deletion in Doubly LL.
04. LRU Cache
05. Cycle in Linkedlist
 - Detect cycle
 - Start point of cycle
 - Break the cycle

Doubly linkedlist

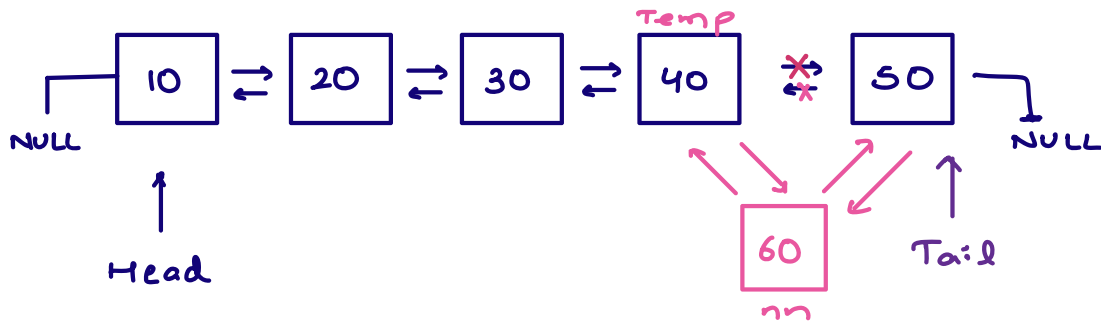
→ A Node having two pointers. one to point towards the next Node, another one to point towards previous node.



01. Insert a new node just before tail of doubly linked list

Note :- Tail reference is given in input

Note :- No. of nodes ≥ 2



```
void insert back ( Node tail , Node nn ) {
```

```
    Node temp = tail.prev
```

```
    nn.next = tail
```

```
    nn.prev = temp
```

```
    temp.next = nn
```

```
    tail.prev = nn
```

```
}
```

TC : $O(1)$

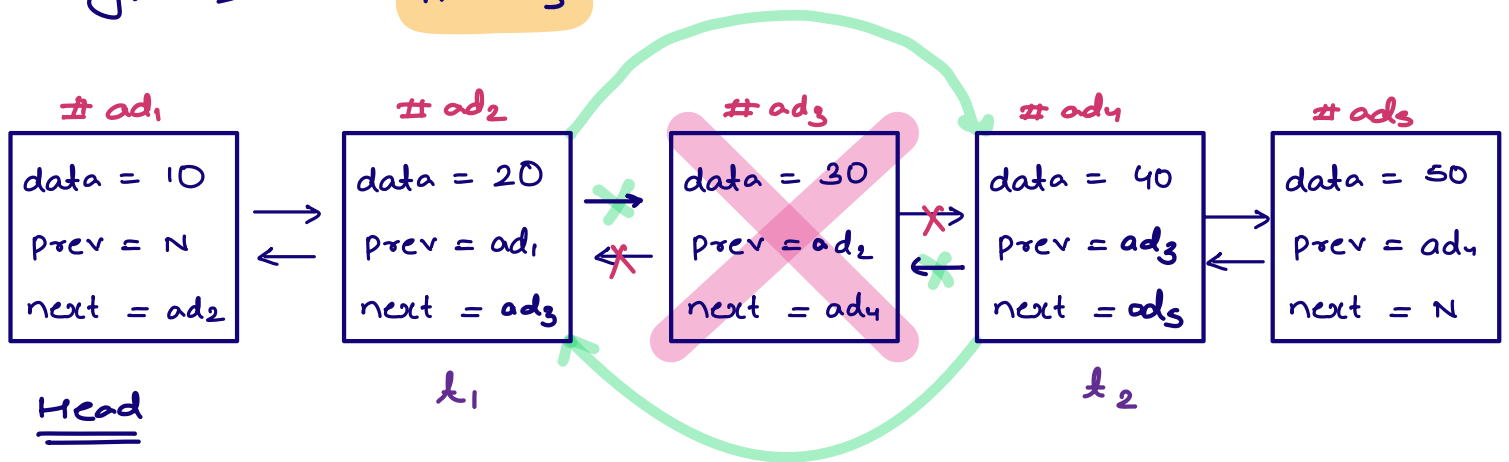
SC : $O(1)$

Q2. Delete a given node from DLL

Note:- Node reference/address is given

Note:- Given node is not head/tail node

Eg:- Delete #ad₃



void deleteNode (Node head, Node temp) ↗ One which needs to be deleted

Node t₁ = temp.prev;

Node t₂ = temp.next;

t₁.next = t₂

t₂.prev = t₁

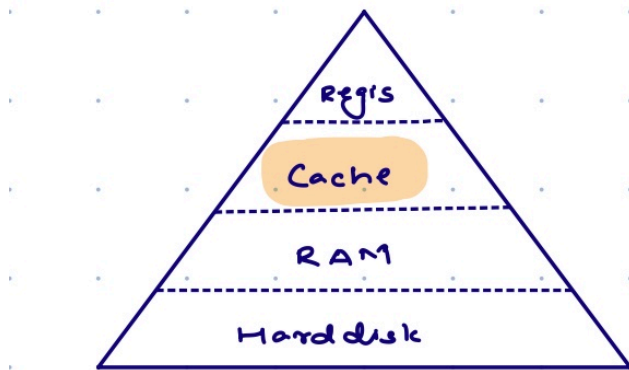
temp.next = Null

temp.prev = Null;

TC: O(1)

SC: O(1)

* Memory Hierarchy



↑
searching
speed
increased

↓
storage
increase

Cache → Limited storage

Maintains most recently used data

There are different ways of implementing cache, one of them is LRU cache.

The Least Recently Used (LRU) Cache operates on the principle that the data most recently accessed is likely to be accessed again in the near future. By evicting the least recently accessed items first, LRU Cache ensures that the most relevant data remains available in the cache.

LRU Cache Memory

Q Given a running stream of Integer & fix memory of size M , we have to maintain the most recent M elements.

Note → If memory is full, delete the least recently used item from the memory.

Data : 7 3 9 2 6 10 12 2

del 7 del 3 del 2
ins 10 ins 12 ins 2

Cache : 5

Most recently used

7 3 9 2 6 10 12 2

Least Recently used

data(x)

Search(x)

Hit

x is present

remove(x)

insert back(x)

Miss

x is not present

size(cache)

Full

delete first();

insert back(x)

Not Full

insert back(x)

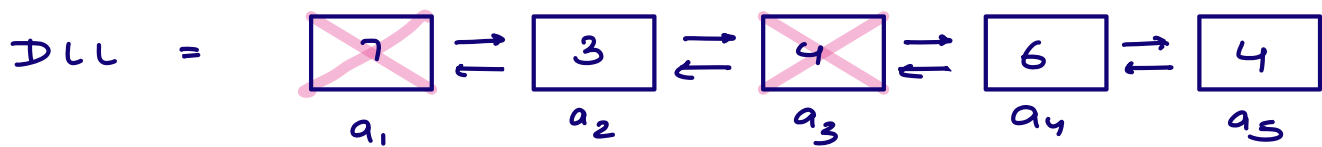
Operations	Array List	Single LL	LL + HS
search(x)	$O(n)$	$O(n)$	$O(1)$
remove(x)	$O(n)$	$O(n)$	$O(n)$ // iterate on LL
insert back(x)	$O(1)$	$O(1)$ tail ptr	$O(1)$
delete first()	$O(n)$	$O(1)$	$O(1)$

Operations	data ↑ address ↑ DLL + HM < Integer, Node >
search(x)	$O(1)$
remove(x)	$O(1)$
insert back(x)	$O(1)$
delete first()	$O(1)$

* Data = 7 3 4 6 4

cache = 3

HashMap = { ~~<7, a₁>~~ <3, a₂> <4, a₃> <6, a₄> }



* Create two dummy nodes as Dummy head & Dummy tail

This will help on focusing more on LRU cache logic & not on maintaining head & tail after first insertion / last deletion / first deletion

class LRUCache

HashMap < I , Node >

Node head = new Node (-1);

Node tail = new Node (-1);

head.next = tail

tail.prev = head

data(x)

Search(x)

Hit
x is present

remove(x)

insert back(x)

Miss
x is not present

size(cache)

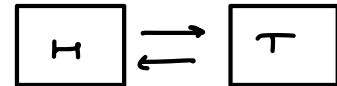
Full

delete first();

insert back(x)

Not Full

insert back(x)



public add(int x, int limit)

if (hm.containsKey(x))

Node temp = hm.get(x) // address of node to del

deleteNode(head, temp);

Node nn = new Node(x);

insertback(tail, nn)

hm.put(x, nn);

else {

if (hm.size() == limit)

Node temp = head.next

deleteNode(head, temp);

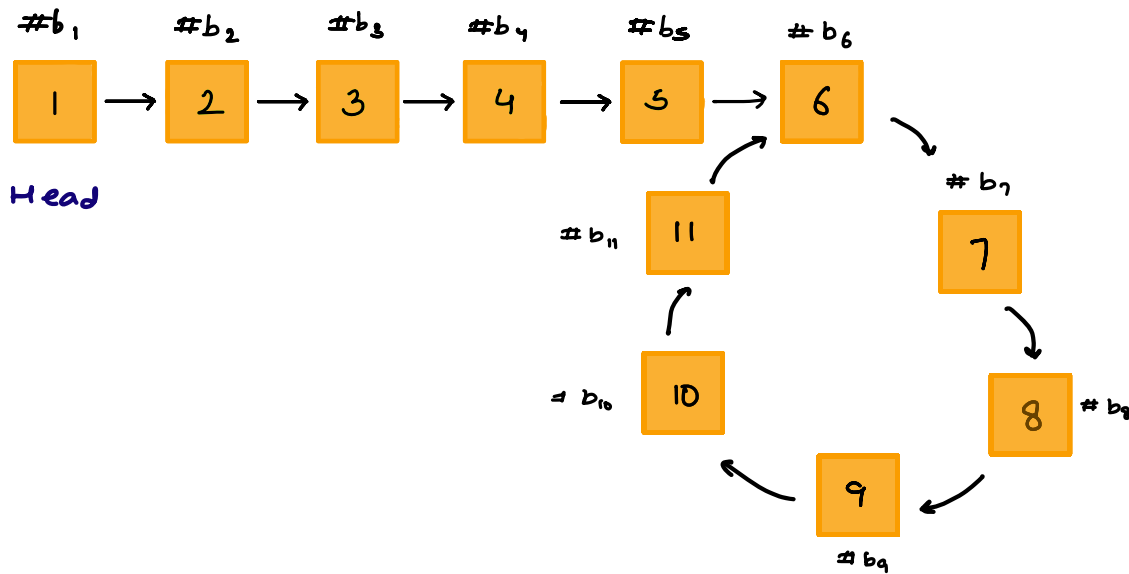
hm.remove(temp.data);

Node nn = new Node(x);

insertback(tail, nn)

hm.put(x, nn);

03. Given a head node of Linkedlist, check for cycle detection?

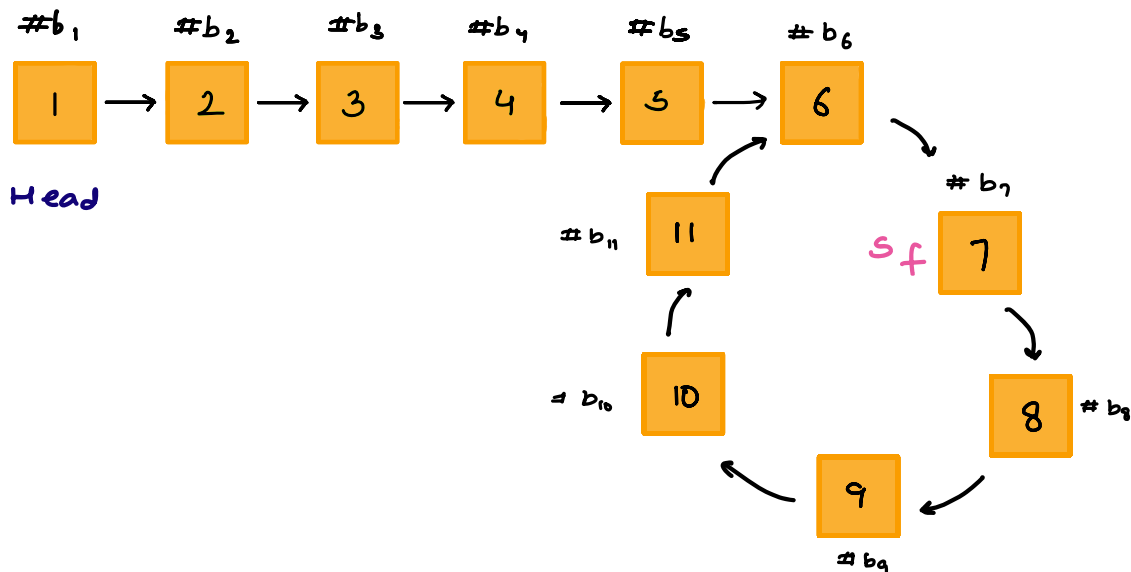


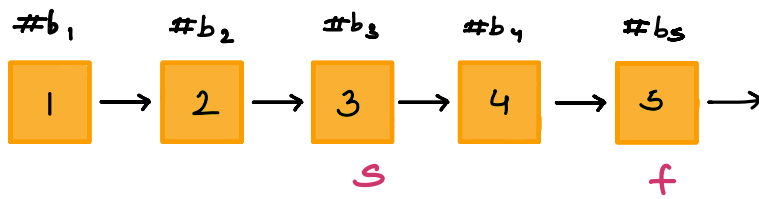
Brute force $\rightarrow$ Create hashset of Node addresses,
if same address is encountered again

TC: $O(N)$
SC: $O(N)$

cycle is present

* No extra space is allowed.





Note → we can go move fast pointer by 3s, 4s, 5s ... but in this case, it is a possibility that fast ptr bypass slow ptr

boolean detectCycle (Node head)

```

1 Node s = f = head;
  while ( f != null && f.next != null )
  {
    s = s.next;
    f = f.next.next;
    if ( s == f ) return true;
  }
2 return false;

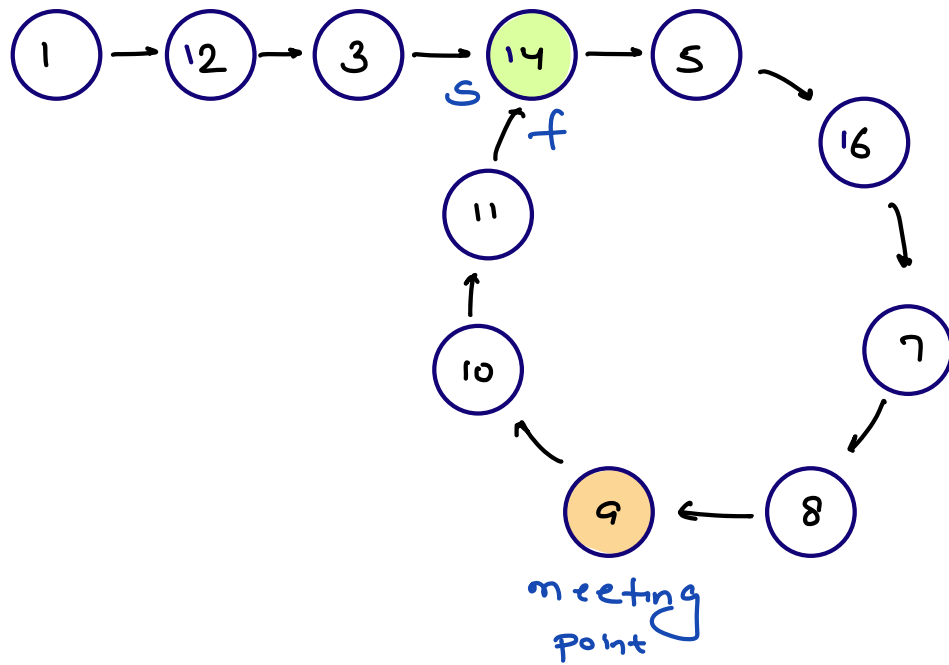
```

Tc: $O(N)$
Sc: $O(1)$

* Remove the cycle

→ Do it on your own

Start point of cycle



01. Detect if cycle is present
02. Move one of the ptr to head
03. Move both slow & fast ptr until they meet.

Node $p_1 = \text{fast}$

Node $p_2 = \text{head}$

while ($p_1 \neq p_2$) {

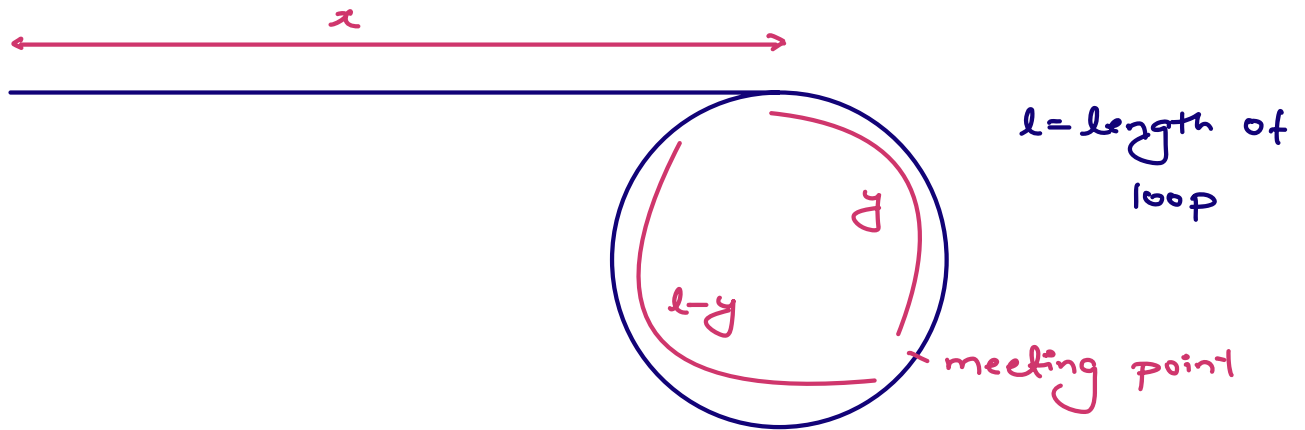
$p_1 = p_1 \cdot \text{next} ;$

$p_2 = p_2 \cdot \text{next} ;$

}

return $p_1 ;$ // start point of cycle

* Mathematical proof for P_1 & P_2 meeting at loop starting



Speed of fast ptr = 2 * Speed of slow ptr

No. of rotations taken by fast ptr = R_1

No. of rotations taken by slow ptr = R_2

Distance covered by fast ptr = $x + R_1 \cdot l + y$

Distance covered by slow ptr = $x + R_2 \cdot l + y$

$$\text{Time} = \frac{\text{Distance}}{\text{speed}}$$

$$\frac{D_f}{S_f} = \frac{D_s}{S_s} \quad \left. \vphantom{\frac{D_f}{S_f} = \frac{D_s}{S_s}} \right\} \begin{array}{l} \text{Meeting time} \\ \text{will be same} \end{array}$$

$$\frac{D_f}{2S_s} = \frac{D_s}{S_s}$$

$$D_f = 2D_s$$

$$x + R_1 l + y = 2 * (x + R_2 l + y)$$

$$x + R_1 l + y = 2x + 2R_2 l + 2y$$

$$R_1 l - 2R_2 l = 2x + 2y - x - y$$

$$R_1 l - 2R_2 l = x + y$$

$$(R_1 - 2R_2) l = x + y$$

$$R_3 l = x + y$$

$$x = R_3 l - y$$

$$R_3 = 1$$

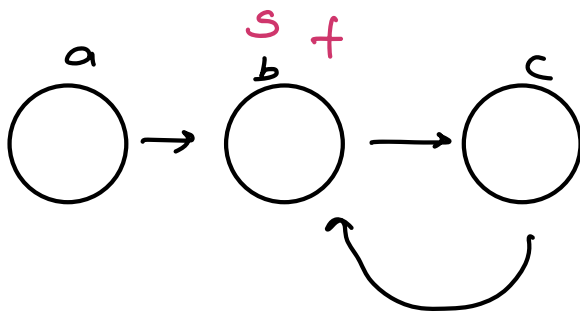
$$x = l - y$$

$$R_3 = 2$$

$$\begin{aligned} x &= 2l - y \\ &= l + (l - y) \end{aligned}$$

$$R_3 = 3$$

$$\begin{aligned} x &= 3l - y \\ &= 2l + (l - y) \end{aligned}$$



$$D_f = 2 D_s$$

$$x + R_1 l + y = 2(x + R_2 l + y)$$

$$x + R_1 l + y = 2x + 2R_2 l + 2y$$

$$R_1 l - 2R_2 l = 2x + 2y - x - y$$

$$\underbrace{(R_1 - 2R_2)l}_{R_3 l} = x + y$$

$$R_3 l = x + y$$

$$\underline{\underline{x = R_3 l - y}}$$

$$R_3 = 1$$

$$x = l - y$$

$$R_3 = 2$$

$$x = 2l - y = l + (l - y)$$

$$R_3 = 3$$

